

Software Design Document (DD)

1. Introduction

1.1 Purpose

The purpose of this Design Document (DD) is to define the high-level architecture, software subsystems, database models, and external interfaces for the client-server urban heat monitoring application. This document establishes a clear transition from the functional and non-functional requirements detailed in our Requirement Analysis and Specification Document (RASD) to the concrete implementation phase.

1.2 Scope

The scope of this system encompasses the implementation of a multi-tiered architecture designed to support city-level heat monitoring, thermal stress indexing, and precipitation analysis for the urban areas of **Milan (Italy)** and **Mexico City (Mexico)**. The software captures, stores, and evaluates three mandatory meteorological metrics: **Temperature** (temp), **Relative Humidity** (rhum), and **Precipitation** (prcp). By structuring these geolocalized historical measurements, the system provides local public authorities and urban planners with descriptive indicators to identify urban heat islands and evaluate climate risks.

2. Architectural Design

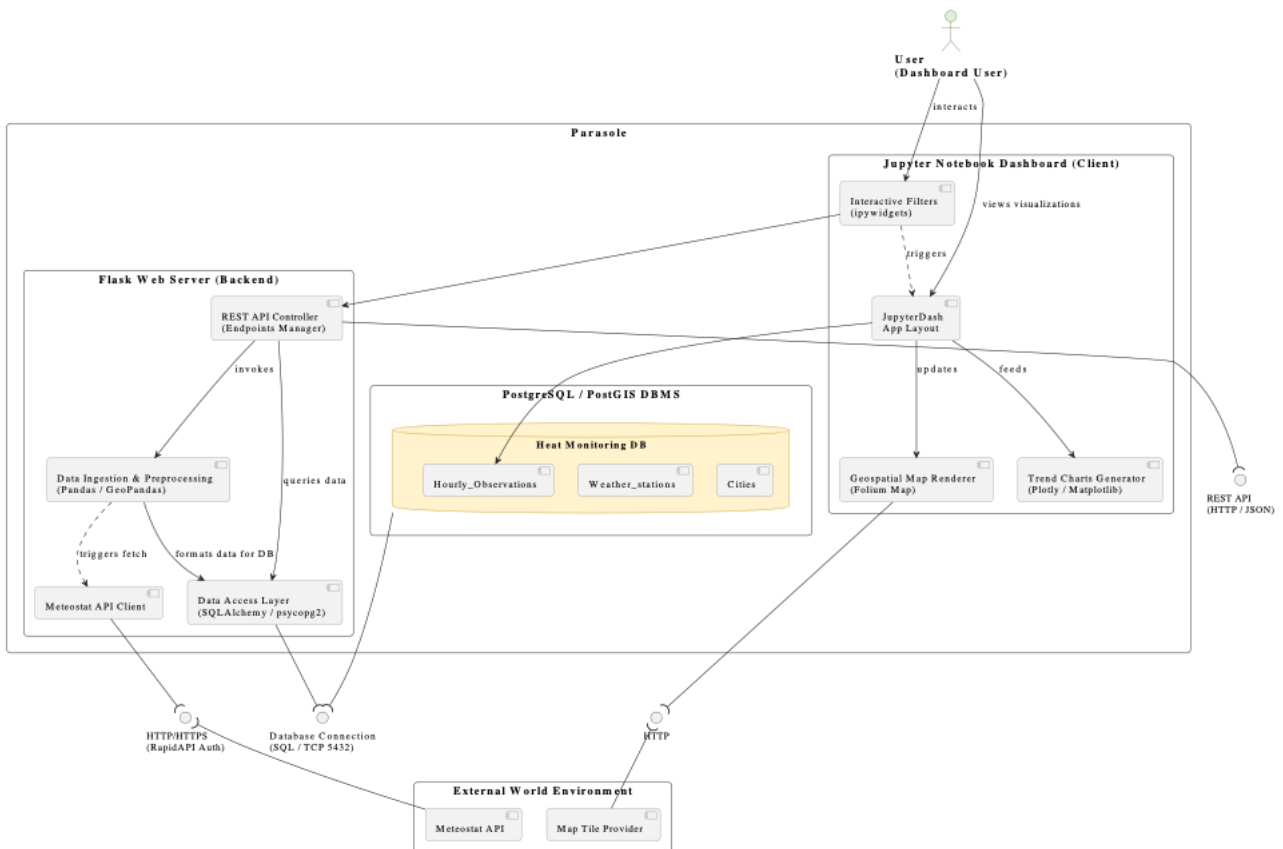
2.1 Overview

The application implements a standard 3-Tier Client-Server Architecture to ensure proper separation of concerns between data persistence, computational logic, and user presentation:

- **Data Tier (DBMS):** A relational database powered by PostgreSQL with the PostGIS spatial extension. It is optimized for storing geolocalized station metadata alongside thousands of volumetric hourly data points.
- **Application Tier (Backend Web Server):** A Python Flask application that acts as the core business controller. It handles synchronous data cleaning, processes database queries, and manages API interaction constraints with external systems.
- **Presentation Tier (Client Dashboard):** An interactive frontend built natively inside a Jupyter Notebook environment. It utilizes rich visualization libraries to render cartographic map layers and responsive time-series graphs.

2.2 Component Diagram (Luca)

The software architecture of the “Parasole” System is modelled using a standard 3-Tier Client-Server structural pattern, operating within a well-defined Machine Boundary to ensure separation of concerns. The runtime interactions between the system components, internal data stores, and external environment infrastructures are detailed in the component diagram below.



Component Description and Subsystems

The application architecture is divided into three functional layers alongside an external world boundary:

1. Presentation Tier (Jupyter Notebook Dashboard)

The client-side environment runs natively inside a Jupyter Notebook workspace and acts as the interactive frontend for the end-user.

- **Jupyter Notebook Dashboard Layout:** Serves as the core UI structural orchestrator that manages the positioning and communication of visual elements across the dashboard grid.
- **Interactive Filters (ipywidgets):** Captures reactive user-driven choices (such as selected target cities, timeline adjustments, or specific parameter filtering criteria) and triggers the downstream pipeline updates.
- **Geospatial Map Renderer (Folium Map):** Processes geolocalized information arrays to render interactive map tile structures and visual heat map overlays tracking urban microclimates.
- **Trend Charts Generator (Plotly / Matplotlib):** Converts continuous statistical data vectors into multi-axis chronological graphs representing temperature, humidity, and rainfall spikes.

2. Application Tier (Flask Web Server)

The backend service acts as the central business controller, decoupling computational data processing routines from data visualization layouts.

- **REST API Controller (Endpoints Manager):** Exposes formal RESTful endpoints to process incoming data inquiries from the presentation client dashboard and returns structured, pre-processed payloads.
- **Data Ingestion & Preprocessing (Pandas / GeoPandas):** Implements server-side logical modules used to manipulate complex data structures. It handles real-time anomalies (such as linear interpolation for corrupt or missing hardware sensor metrics) before record updates.
- **Meteostat API Client:** Manages external server communications, structuring specific query formats and enforcing authorization parameters to safely pool weather records when a local data missing gap is encountered.
- **Data Access Layer (SQLAlchemy / psycopg2):** Provides an object-relational abstraction layer to safely translate operational logic into transactional query commands, insulating the web server from raw schema execution structures.

3. Data Tier (PostgreSQL / PostGIS DBMS)

The storage subsystem relies on a relational model enhanced with spatial attributes to efficiently archive time-series metrics.

- **Heat Monitoring DB:** Contains the normalized database tables required for spatial and chronological query executions:
 - *City Table:* Maintains the top-level administrative and time zone parameters for the target metropolitan areas (Milan and Mexico City).
 - *WeatherStation Table:* Stores spatial geometry coordinates (using the native EPSG:4326 Point system) and physical metadata for localized instrumentation nodes.
 - *Hourly Observation Table:* Contains expanding records of environmental variables (temperature, relative humidity, and precipitation) captured at regular hourly intervals.

Runtime Interfaces and Communication Protocols

As depicted in the diagram, system orchestration is sustained through four primary communication paths using standard network interfaces:

- **REST API Interface (HTTP/JSON):** The Presentation Tier interacts with the Application Tier over stateless HTTP network connections. The dashboard sends structural GET requests to the backend server endpoints, which returns responses encoded entirely as lightweight JSON data payloads.

- **Database Connection (SQL/TCP 5432):** The Data Access Layer establishes a secure database connection to the PostgreSQL engine using standard TCP/IP routing on port 5432. Communications include optimized SQL relational statements and geometric PostGIS operations.
- **Meteostat Client Interface (HTTPS/RapidAPI):** The server-side client communicates with the external Meteostat remote architecture through encrypted HTTPS connections. Requests utilize formal authentication headers validated against RapidAPI Gateway profiles to access historical weather sensor archives.
- **Map Tile Protocol (HTTP/OpenStreetMap):** The geospatial rendering component running inside the client dashboard contacts external third-party Map Tile Providers over standard HTTP channels to fetch real-time geographic map tile segments dynamically.

3. Software Design & Database Design (Luca)

This chapter details the logical and physical data architecture of the “Parasole” System. The system design is structured to guarantee strict integrity, optimize the retrieval of high-volume time-series vectors, and properly handle geospatial attributes using dedicated DBMS extensions.

3.1 Relational Database Schema

The persistence subsystem implements a normalized relational schema which eliminates data redundancy and optimizes the execution path of chronological aggregation queries. Spatial variables are natively managed through the **PostGIS** extension, enabling advanced indexing of geographical coordinates assigned to individual weather instrumentation nodes.

The formal structure of the SQL tables composing the Heat Monitoring DB is defined below:

1. Table: Cities

This registry catalogue maintains high-level administrative parameters for the target metropolitan areas monitored by the application (Milan and Mexico City).

- **city_id** (*Integer, Primary Key, Serial*): Auto-incrementing identifier generated by the DBMS to uniquely distinguish each city.
- **name** (*Varchar(100), Not Null, Unique*): The distinct name of the targeted city (e.g., "Milan", "Mexico City").
- **country** (*Varchar(100), Not Null*): The name of the corresponding nation for filtering and administrative classification purposes.
- **timezone** (*Varchar(50), Not Null*): A standard administrative timezone mapping string (e.g., Europe/Rome, America/Mexico_City), essential for handling regional chronological adjustments during data visualization.

2. Table: Weather_Stations

Tracks physical instrumentation nodes deployed across the urban territory.

- **station_id** (*Varchar(50), Primary Key*): Unique alphanumeric identifier globally assigned to the sensor block by the Meteostat repository.
- **name** (*Varchar(150), Not Null*): Localized descriptive name of the station or its monitoring vicinity.
- **elevation** (*Float, Nullable*): Vertical elevation above sea level measured in meters.
- **geom** (*Geometry(Point, 4326), Not Null*): PostGIS spatial attribute tracking Longitude and Latitude. It uses the native **WGS 84 (EPSG:4326)** coordinate system, enabling optimized spatial operations and real-time mapping layers on the Folium canvas.
- **city_id** (*Integer, Foreign Key*): References the matching entry in the cities table, enforcing referential integrity.

3. Table: Hourly_Observations

Represents the volumetric time-series data table, designed to archive individual environmental variables captured at regular hourly intervals by each active sensor.

- **observation_id** (*BigInt, Primary Key, Serial*): Unique row identifier for the records.
- **station_id** (*Varchar(50), Foreign Key*): Foreign key linking the telemetry row directly back to its parent sensor block in the weather_stations table.
- **record_time** (*Timestamp with Time Zone, Not Null*): The absolute timestamp of the observation, normalized to the **UTC** timezone at the storage layer to prevent chronological synchronization anomalies across distant geographical regions.
- **temperature** (*Float, Nullable*): Measured air temperature reading expressed in degrees Celsius (°C).
- **rel_humidity** (*Float, Nullable*): Relative air humidity percentage (%).
- **precipitation** (*Float, Nullable*): Accumulated column height of liquid precipitation recorded over the preceding hour, measured in millimeters (mm).

Table	Usage	Attributes
Cities	Maintains high-level administrative parameters for the target metropolitan areas monitored by the application (Milan and Mexico City).	city_id; name; country; timezone
Weather_Stations	Tracks physical instrumentation nodes deployed across the urban territory.	station_id; city_id; name; elevation;

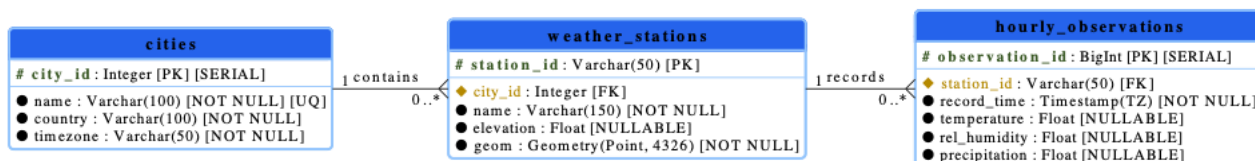
		geom
<i>Hourly_Observations</i>	Represents the volumetric time-series data table, designed to archive individual environmental variables captured at regular hourly intervals by each active sensor.	observation_id; station_id; record_time; temperature; rel_humidity; precipitation

3.2 Entity-Relationship Diagram (ERD)

The relationship matrix guarantees systemic integrity, mapping single stable geographic configurations across multiple independent, expanding time-series observations.

The cardinality metrics are structured as follows:

- A single **Cities** entry can be linked to one or many (**1:N**) **Weather_Station** blocks. Conversely, each station must belong to exactly one administrative city.
- A single **Weather_Stations** block produces one to many (**1:N**) **Hourly_Observation** telemetry rows over time. Each observation row is indissolubly tied back to a single originating sensor node.



3.3 Data Integrity & Indexing Strategy

To ensure that the Jupyter presentation client dashboard renders geospatial layers and trend charts responsively without execution latencies, specific query optimization strategies are implemented at the database tier:

1. **Spatial Indexing (GiST):** A Generalized Search Tree (GiST) index is applied directly to the geom column within the weather_stations table. This allows the Flask application tier to instantly resolve geometric constraints and coordinate station alignments against city boundaries.
2. **Composite Time-Series Indexing:** Since backend analytical queries filter weather records using a bounding date window for a targeted node, a composite B-Tree index is deployed on the (station_id, record_time) columns within the hourly_observations table. This drastically eliminates the need for expensive sequential table scans.

4. REST API Specification (Dario)

The Application Tier acts as the central software orchestrator of the **Parasole** system. By exposing a stateless, structured RESTful API, the Flask server completely decouples

presentation elements from the physical database execution paths and the external environment network requests.

All data transactions exchanged between the Jupyter Notebook client dashboard and the backend server are transmitted over standard HTTP transport protocols using the standard application/json content-type media header. To prevent cross-origin blocks during client execution, Cross-Origin Resource Sharing (CORS) is explicitly enabled on the server configuration.

4.1 Global API Configuration & Status Codes

- **Base URL Path:** All endpoints are standardized under the resource versioning path: /api/v1
- **Timezone Standard:** To avoid temporal synchronization inconsistencies between Milan and Mexico City, all input and output datetime fields are handled and formatted using the **UTC timezone** standard (ISO 8601 extended layout).
- **HTTP Status Codes Protocol:** The API Controller handles server responses utilizing specific HTTP status structures:
 - 200 OK: Request successfully executed, payload attached.
 - 400 Bad Request: Mandatory query parameters are missing or malformed.
 - 404 Not Found: The requested resource (city, station, or observation range) does not exist.
 - 503 Service Unavailable: External Meteostat API usage constraints or network failures prevent data pooling.

4.2 Endpoints Directory

1. Fetch Station Coordinates

- **HTTP Method:** GET
- **Path:** /api/v1/stations
- **Description:** Retrieves metadata parameters and geolocations for all operational weather stations associated with a target metropolitan city. This endpoint enables the client-side geospatial map renderer to position station markers correctly on the Folium canvas.
- **Query Parameters:**

Parameter	Type	Required	Description	Example
name	String	Yes	Name of the targeted city to filter available stations.	?name=Milan

- **Backend Execution Workflow:**

The REST API Controller extracts the name parameter and invokes the Data Access Layer. It runs an optimized SQL relational query that joins the Cities and Weather_Stations tables. PostGIS point geometries stored in the geom column are unwrapped into standard decimal Latitude and Longitude values prior to serialization.

- **Response Payload (JSON Format - 200 OK):**

JSON

```
[
{
  "station_id": "16080",
  "name": "Milano Linate",
  "elevation": 107.0,
  "latitude": 45.445,
  "longitude": 9.276
}
```

2. Query Historical Logs

- **HTTP Method:** GET
- **Path:** /api/v1/weather
- **Description:** Extracts clean timeseries vectors tracking temperature, relative humidity, and precipitation arrays over an explicit bounding date window for a single sensor station.
- **Query Parameters:**

Parameter	Type	Required	Description	Example
station_id	String	Yes	Unique alphanumeric global key of the station.	16080
start_date	String	Yes	Bounding window start date formatted as YYYY-MM-DD.	2026-06-01

end_date	String	Yes	Bounding window end date formatted as YYYY-MM-DD.	2026-06-07
----------	--------	-----	---	------------

- **Backend Caching & Lazy-Loading Enforcement:**

Upon receiving the parameters, the server performs a local database lookup within the Hourly_Observations table.

- **Cache Hit Strategy:** If the requested date range is fully present in local storage, the rows are structured and returned to the client in less than 2 seconds. No external call is spent.
- **Cache Miss Strategy:** If data is missing, the backend triggers the Meteostat API Client submodule to query the remote RapidAPI server endpoint. The ingested raw rows are passed to the Data Ingestion & Preprocessing layer, where Pandas drops corrupted elements or performs synchronous linear interpolation on missing attributes. The resulting clean dataset is committed to the local SQL repository for future cache calls and returned to the dashboard client.

- **Response Payload (JSON Format - 200 OK):**

JSON

```
{
  "station_id": "16080",
  "city": "Milan",
  "parameters": ["temp", "rhum", "prcp"],
  "series": [
    {
      "timestamp": "2026-06-01T12:00:00Z",
      "temp": 28.5,
      "rhum": 55.0,
      "prcp": 0.0
    }
  ]
}
```

4.3 Client-Side Error Response Framework

To ensure robust interaction, any failure path triggered inside the server returns a explicit JSON block tracking the error root cause:

- **Scenario A: Missing or Malformed Filter Parameters (400 Bad Request)**

JSON

```
{  
  "error": "Bad Request",  
  "message": "Missing mandatory query parameters: station_id, start_date, and end_date must be provided.",  
  "status": 400  
}
```

- **Scenario B: Remote RapidAPI Rate Quota Depleted (503 Service Unavailable)**

If a cache miss occurs but the system has exhausted its monthly 500-request quota limit on Meteostat, the API engine blocks network errors and returns an alert informing the client that only local cached rows can be queried.

JSON

```
{  
  "error": "Service Unavailable",  
  "message": "Meteostat API monthly rate quota exceeded. Remote historical logs are currently unavailable; only locally cached records can be retrieved.",  
  "status": 503  
}
```

5. Meteostat API Integration (Fernando)

5.1 Meteostat API Client

Given the subscription profile configured on RapidAPI restricting to an absolute ceiling of **500 free requests per month**, it is necessary to optimize application uptime and prevent quota exhaustion. For this reason, the Jupyter Notebook client interface is not in contact with Meteostat endpoints directly. The strategy then consists of two layers: a Meteostat API Client layer to perform the calls according to the API documentation and the data ingestion and preprocessing layer that orchestrates an intermediary lazy-loading data flow.

The intermediary layer for the Meteostat API integration consists of 4 methods:

- **Hourly Records:** Provides the records per hour, per station of a period no greater than 30 days. These records are stored on the database to avoid quickly surpassing the API request limit.
- **Station Metadata:** Provides the name, location and status along with certain other parameters of each weather station. This endpoint is not consuming resources directly from the API, since it's in a different endpoint of the Meteostat API.
- **Station Normals:** Provide a set of descriptive statistics of a reduced set of weather parameters calculated from a 30-year reference period. Useful to compare and determine extreme weather events.
- **Nearby Stations:** Provides a list of the stations found in a radius of 50 kilometers around the center of each city.

5.2 Continuous data ingestion strategy and caching logic

The Data Ingestion and Preprocessing layer takes care of the lazy-loading system. The backend calls the stations/endpoint only when the data is not found on the persistent Postgres database. This helps reducing the number of calls, lowering the cost of the system.

6. Interactive Dashboard Design (Juanita)

The Presentation Tier is delivered as an analytical client dashboard embedded inside a Jupyter Notebook workspace. It communicates exclusively with the Application Tier (Flask Backend) through HTTP REST API calls and renders interactive maps, charts, and alert panels for urban heat monitoring analysis.

6.1 Dashboard Architecture

The dashboard operates as the third layer of the 3-Tier Client-Server Architecture. It does not connect directly to the database or to the Meteostat API. All data access is mediated through the Flask backend via REST API endpoints.

From	To	Protocol
Dashboard	Flask Backend	HTTP GET requests via Python requests library. Responses in JSON format.
Dashboard	OpenStreetMap	HTTP tile requests via Folium (automatic, handled by the library).

6.2 Layout and Interface Components

- **Global Filters Component (Jupyter Widgets):** Integrates selection controls that allow users to select either Milan or Mexico City, choose a weather parameter, and define a date range using date pickers. A search bar allows filtering stations by name within the selected city.

- **Geospatial Map Canvas (Folium / OpenStreetMap):** Generates interactive geographic layers showing the location of weather stations. It transforms raw meteorological readings into visual heatmap overlays, allowing users to explore spatial patterns and compare microclimate variations across the selected area.
- **Temporal Trend Canvas (Plotly):** Renders interactive dual-axis line charts tracking the relationships between temperature spikes and relative humidity levels. This enables users to visualize the real-world occurrence of urban heat island effects during summer periods.
- **Heat Stress Watch Engine (Threshold Panel):** Allows users to set temperature and humidity thresholds using slider controls (pre-defined defaults: Temp > 35°C, Humidity > 70%). The panel automatically highlights periods where both thresholds are exceeded and displays an alert banner with color-coded severity levels.

6.3 Data Flow per Use Case

UC2 - Visualize Geospatial Weather Maps

The user selects a city from the dashboard filters. The dashboard requests station metadata from the backend, which returns the list of stations with their coordinates. The map component renders an interactive map centered on the selected city with markers representing each weather station. Users can click on markers to view station details.

UC3 - Analyze Meteorological Trends

The user selects a station, a weather parameter, and a date range. The dashboard requests the corresponding hourly observations from the backend. The chart component renders an interactive dual-axis line chart showing the temporal evolution of the selected parameter. Users can hover, zoom, and pan to explore the data.

UC4 - Configure and Monitor Heat Stress Alerts

The dashboard loads with pre-defined alert thresholds (e.g. Temperature > 30°C, Humidity > 70%). An alert banner automatically indicates the severity level for the selected city. Users can adjust the thresholds using slider controls, and the alert panel updates instantly to highlight periods where both conditions are exceeded. A downloadable report can be exported for sharing with civil protection agencies and citizens.

6.4 Summary Indicators

The dashboard computes and displays descriptive statistics for the selected city and time range, including average, minimum, and maximum temperature; average relative humidity; total accumulated precipitation; number of hours exceeding the configured alert thresholds; and the count of active stations reporting data. These indicators provide a quick overview of the meteorological conditions without requiring the user to inspect the charts in detail.

6.4 Error Handling

The dashboard handles the following error conditions:

Condition	Dashboard Response
Backend unreachable	Displays a warning message indicating the server is not responding.
No data available	Displays an informative message and shows an empty state in the chart area.
Map tiles unavailable	Station data is displayed in table format as a fallback.
Invalid threshold input	Slider controls enforce valid ranges, preventing invalid configurations.
API rate limit exceeded	Displays a message indicating only locally cached data is available.

6.5 Wireframe Layout

The block layout below outlines the organization of interactive elements across the screen, ensuring a consistent and structured workflow for the end-user.

Global filters City selector / Search station / Date range / Load Data button	
Alert banner- automatically shows severity level (Critical / Warning / Normal)	
Geospatial maps Interactive map with station markers	Trend charts Interactive line charts with threshold line
Heat stress level Threshold sliders - Critical periods table - Export Report button	Summary indicators Avg/Min/Max temp - Humidity - Precipitation - Critical hours